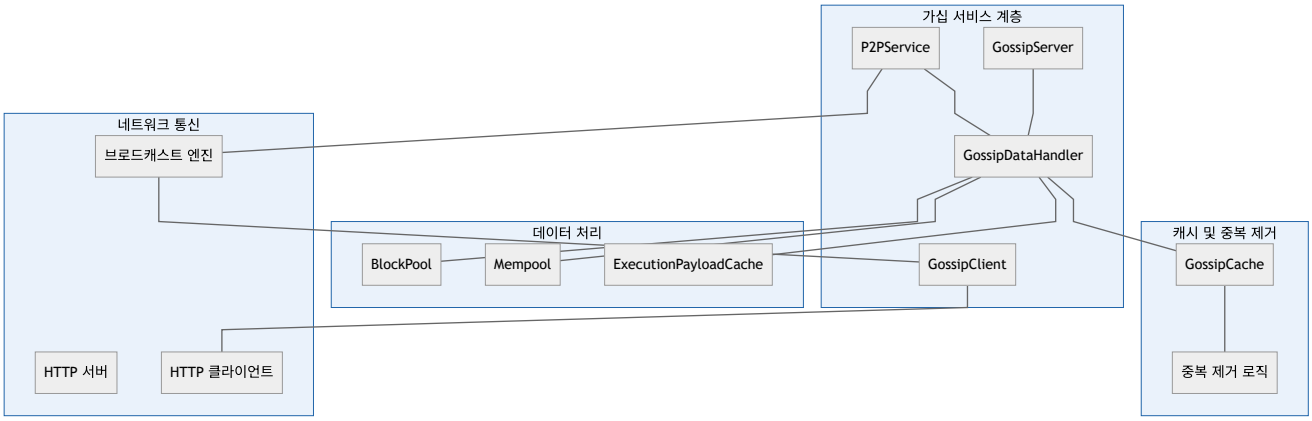
가십 프로토콜 구현 상세 분석

개요

Irys 체인의 가십 프로토콜은 분산 네트워크에서 데이터를 효율적으로 전파하기 위한 핵심 메커니즘입니다. 이 프로토콜은 블록, 트랜잭션, 청크, 실행 페이로드 등 다양한 유형의 데이터를 네트워크 전체에 안정적으로 전파하며, 중복 전송을 방지하고 네트워크 파티션 상황에서도 복구 능력을 제공합니다.

가십 프로토콜 아키텍처



가십 메시지 구조

1. 기본 가십 요청 구조

```
pub struct GossipRequest<T> {
pub miner_address: Address, // 송신자의 마이닝 주소
pub data: T, // 실제 데이터 페이로드
}
```

2. 가십 데이터 유형

```
pub enum GossipData {
Chunk(Arc<UnpackedChunk>), // 데이터 청크
Transaction(Arc<DataTransactionHeader>), // 데이터 트랜잭션
CommitmentTransaction(Arc<CommitmentTransaction>), // 커밋먼트 트랜잭션
Block(Arc<IrysBlockHeader>), // 블록 헤더
ExecutionPayload(Block), // EVM 실행 페이로드
IngressProof(Arc<IngressProof>), // 인그레스 증명
}
```

3. 데이터 요청 구조

```
pub enum GossipDataRequest {
Block(BlockHash), // 블록 요청
ExecutionPayload(B256), // 실행 페이로드 요청
```

```
Chunk(ChunkPathHash), // 청크 요청
}
```

가십 서버 구현

1. HTTP 엔드포인트

가십 서버는 다음과 같은 REST API 엔드포인트를 제공합니다:

```
web::scope("/gossip")
.route("/transaction", web::post().to(Self::handle_transaction))
.route("/commitment_tx", web::post().to(Self::handle_commitment_tx))
.route("/chunk", web::post().to(Self::handle_chunk))
.route("/block", web::post().to(Self::handle_block))
.route("/ingress_proof", web::post().to(Self::handle_ingress_proof))
.route("/execution_payload", web::post().to(Self::handle_execution_payload))
.route("/get_data", web::post().to(Self::handle_data_request))
.route("/pull_data", web::post().to(Self::handle_pull_data))
.route("/health", web::get().to(Self::handle_health_check))
```

2. 피어 검증 시스템

IP 주소 기반 인증:

```
fn check_peer(
peer_list: &PeerList,
```

```
req: &actix_web::HttpRequest,

miner_address: Address,

) -> Result<PeerListItem, HttpResponse> {

let Some(peer_address) = req.peer_addr() else {

return Err(HttpResponse::BadRequest().finish());

};



if  let Some(peer) = peer_list.peer_by_mining_address(&miner_address) {

if  peer.address.gossip.ip() != peer_address.ip() {

debug!("IP mismatch: expected {}, got {}",

peer.address.gossip.ip(), peer_address.ip());

return Err(HttpResponse::Forbidden().finish());

}

Ok(peer)

} else {

warn!("Unknown miner address: {}", miner_address);

Err(HttpResponse::Forbidden().finish())

}

}
```

3. 가십 수신 제어

동적 수신 제어:

```
if !server.data_handler.sync_state.is_gossip_reception_enabled() {
warn!("Gossip reception is disabled, ignoring data");
return HttpResponse::Forbidden().finish();
}
```

데이터 전파 알고리즘

1. 브로드캐스트 전략

1.1 피어 선택 알고리즘

```
const MAX_PEERS_PER_BROADCAST: usize = 5;
const BROADCAST_INTERVAL: Duration = Duration::from_secs(1);

async fn broadcast_data(
&self,
broadcast_message: GossipBroadcastMessage,
peer_list: &PeerList,
) -> GossipResult<()> {
let mut peers = peer_list.top_active_peers(None, None);
peers.shuffle(&mut rand::thread_rng());

while !peers.is_empty() {
// 이미 데이터를 본 피어들 제외
```

```
let peers_that_seen_data = self.cache.peers_that_have_seen(&key)?;
peers.retain(|(peer_miner_address, _peer)| {
!peers_that_seen_data.contains(peer_miner_address)
});

if peers.is_empty() {
break;
}

let n = std::cmp::min(MAX_PEERS_PER_BROADCAST, peers.len());
let selected_peers = peers.get(0..n);

// 선택된 피어들에게 데이터 전송
for (peer_miner_address, peer_entry) in selected_peers {
self.client.send_data_and_update_the_score_detached(
(peer_miner_address, peer_entry),
Arc::clone(&broadcast_data),
peer_list,
);
}

tokio::time::sleep(BROADCAST_INTERVAL).await;
}
```

```
}
```

1.2 점진적 전파

단계별 브로드캐스트:

- 각 라운드에서 최대 5개 피어에게만 전송
- 1초 간격으로 다음 라운드 진행
- 이미 데이터를 본 피어는 제외
- 모든 활성 피어가 데이터를 받을 때까지 반복

2. 풀 기반 데이터 요청

2.1 다중 피어 요청 전략

```
pub  async  fn  pull_data_from_network<T>(
&self,
data_request: GossipDataRequest,
use_trusted_peers_only: bool,
peer_list: &PeerList,
map_data: fn(GossipData) -> Result<T, PeerNetworkError>,
) -> Result<(Address, T), PeerNetworkError> {
let  mut  peers = if  use_trusted_peers_only {
peer_list.trusted_peers()
} else {
peer_list.top_active_peers(Some(10), None)
```

```
};

peers.shuffle(&mut rand::thread_rng());
peers.truncate(5);

// 최대 5번의 시도로 여러 피어에서 데이터 요청
for attempt in 1..=5 {
for peer in &peers {
match self.pull_data_and_update_the_score(peer, data_request.clone(), peer_list
Ok(Some(data)) => {
return Ok((*peer.0, map_data(data)?));
}
Ok(None) => continue, // 피어가 데이터를 가지고 있지 않음
Err(err) => continue, // 다음 피어 시도
}
}
tokio::time::sleep(Duration::from_millis(100)).await;
}

Err(PeerNetworkError::FailedToRequestData(format!(
"Failed to fetch {:?} after trying 5 peers", data_request
)))
```

```
}
```

메시지 중복 제거 및 캐싱 전략

1. 가십 캐시 구조

```
pub struct GossipCache {
chunks: Cache<ChunkPathHash, Arc<RwLock<HashSet<Address>>>>,
transactions: Cache<IrysTransactionId, Arc<RwLock<HashSet<Address>>>>,
blocks: Cache<BlockHash, Arc<RwLock<HashSet<Address>>>>,
payloads: Cache<B256, Arc<RwLock<HashSet<Address>>>>,
ingress_proofs: Cache<H256, Arc<RwLock<HashSet<Address>>>>,
}
```

2. 중복 제거 메커니즘

2.1 데이터 수신 시 중복 검사

```
pub async fn handle_transaction(
&self,
transaction_request: GossipRequest<DataTransactionHeader>,
) -> GossipResult<()> {
let tx_id = transaction_request.data.id;
let source_miner_address = transaction_request.miner_address;
```

```
// 캐시에서 중복 검사
let already_seen = self.cache.seen_transaction_from_any_peer(&tx_id)?;
// 소스 피어를 캐시에 기록
self.cache.record_seen(source_miner_address, GossipCacheKey::Transaction(tx_id))

if already_seen {
debug!("Transaction {} already seen, skipping", tx_id);
return Ok(());
}

// 멤풀에서 중복 검사
if self.mempool.is_known_transaction(tx_id).await? {
debug!("Transaction already handled, skipping");
return Ok(());
}

// 새로운 트랜잭션 처리
self.mempool.handle_data_transaction_ingress(transaction_request.data).await?;
Ok(())
}
```

2.2 TTL 기반 캐시 관리

```rust
const GOSSIP_CACHE_TTL: Duration = Duration::from_secs(300); // 5분


impl GossipCache {
pub fn new() -> Self {
Self {
chunks: Cache::builder().time_to_live(GOSSIP_CACHE_TTL).build(),
transactions: Cache::builder().time_to_live(GOSSIP_CACHE_TTL).build(),
blocks: Cache::builder().time_to_live(GOSSIP_CACHE_TTL).build(),
payloads: Cache::builder().time_to_live(GOSSIP_CACHE_TTL).build(),
ingress_proofs: Cache::builder().time_to_live(GOSSIP_CACHE_TTL).build(),
}
}
}
```

3. 피어별 데이터 추적

피어가 본 데이터 추적:

```rust
pub fn record_seen(
&self,
miner_address: Address,
key: GossipCacheKey,
) -> GossipResult<()> {
match key {
```

```
GossipCacheKey::Transaction(tx_id) => {
let peer_set = self.transactions.get(&tx_id).unwrap_or_else(|| {
let new_set = Arc::new(RwLock::new(HashSet::new()));
self.transactions.insert(tx_id, new_set.clone());
new_set
});
peer_set.write().unwrap().insert(miner_address);
}
// 다른 데이터 유형들도 동일한 패턴
}
Ok(())
}
```

네트워크 파티션 대응 및 복구 메커니즘

1. 파티션 감지

1.1 피어 연결 상태 모니터링

```
// 비활성 피어 헬스 체크
ctx.run_interval(INACTIVE_PEERS_HEALTH_CHECK_INTERVAL, |act, ctx| {
let inactive_peers = act.peer_list.inactive_peers();
for (mining_addr, peer) in inactive_peers {
let client = act.gossip_client.clone();
```

```
let fut = async move { client.check_health(peer_address).await }
.into_actor(act)
.map(move |result, act, _ctx| match result {
Ok(true) => {
act.increase_peer_score(&mining_addr, ScoreIncreaseReason::Online);
}
Ok(false) => {
act.decrease_peer_score(&mining_addr, ScoreDecreaseReason::Offline);
}
Err(e) => {
error!("Health check failed for peer {:?}: {:?}", mining_addr, e);
}
});
ctx.spawn(fut);
}
});
```

2. 복구 메커니즘

2.1 자동 재연결

피어 재발견 및 연결:

- 정기적인 헬스 체크를 통한 피어 상태 확인
- 오프라인 피어의 자동 재연결 시도

- 새로운 피어 발견을 통한 네트워크 확장

2.2 데이터 동기화

누락된 데이터 복구:

```
// 블록 동기화를 통한 데이터 복구
pub async fn pull_and_process_block(
&self,
block_hash: BlockHash,
use_trusted_peers_only: bool,
) -> GossipResult<()> {
let (source_address, irys_block) = self
.gossip_client
.pull_block_from_network(block_hash, use_trusted_peers_only, &self.peer_list)
.await?;


// 블록과 함께 누락된 트랜잭션들도 함께 가져오기
let mut missing_tx_ids = Vec::new();
for tx_id in block_header.data_ledgers.iter().flat_map(|ledger| ledger.tx_ids
if !self.is_known_tx(tx_id).await? {
missing_tx_ids.push(tx_id);
}
}
```

```
// 누락된 트랜잭션들을 소스 피어에서 가져오기
let missing_txs = self
.api_client
.get_transactions(source_api_address, &missing_tx_ids)
.await?;


// 트랜잭션들을 멤풀에 추가
for tx_response in missing_txs {
self.mempool.handle_transaction_ingress(tx_response).await?;
}


Ok(())
}
```

데이터 유형별 처리 로직

1. 블록 처리

1.1 블록 검증 및 처리

```
pub async fn handle_block_header(
&self,
block_header_request: GossipRequest<IrysBlockHeader>,
source_api_address: SocketAddr,
```

```
data_source_ip: SocketAddr,

) -> GossipResult<()> {

let  block_header = block_header_request.data;

let  block_hash = block_header.block_hash;



// 동기화 범위 확인

if  self.sync_state.is_syncing()

&& block_header.height > (self.sync_state.sync_target_height() + 1) as u64 {

debug!("Block {} is out of sync range, skipping", block_hash);

return Ok(());

}



// 서명 검증

if !block_header.is_signature_valid() {

warn!("Block {} has invalid signature", block_hash);

self.peer_list.decrease_peer_score(&source_miner_address, ScoreDecreaseReason::B

return Err(GossipError::InvalidData(InvalidDataError::InvalidBlockSignature));

}



// 신뢰할 수 있는 피어에서 온 블록인지 확인

let  skip_block_validation = self.sync_state.is_syncing_from_a_trusted_peer()

&& self.sync_state.is_in_trusted_sync_range(block_header.height as usize)

&& self.peer_list.is_a_trusted_peer(source_miner_address, data_source_ip.ip());
```

```
// 블록 풀에서 처리
self.block_pool.process_block(Arc::new(block_header), skip_block_validation).awa
Ok(())
}
```

2. 트랜잭션 처리

2.1 데이터 트랜잭션

```
pub  async  fn  handle_transaction(
&self,
transaction_request: GossipRequest<DataTransactionHeader>,
) -> GossipResult<()> {
let  tx = transaction_request.data;
let  tx_id = tx.id;

// 중복 검사
if  self.cache.seen_transaction_from_any_peer(&tx_id)? {
return Ok(());
}

if  self.mempool.is_known_transaction(tx_id).await? {
return Ok(());
```

```
}

// 멤풀에 전송
match self.mempool.handle_data_transaction_ingress(tx).await {
Ok(()) | Err(TxIngressError::Skipped) => Ok(()),
Err(error) => Err(GossipError::from(error)),
}
}
```

3. 실행 페이로드 처리

```
pub async fn handle_execution_payload(
&self,
execution_payload_request: GossipRequest<Block>,
) -> GossipResult<()> {
let evm_block = execution_payload_request.data;
let sealed_block = evm_block.seal_slow();
let evm_block_hash = sealed_block.hash();

// 중복 검사
let payload_already_seen = self.cache.seen_execution_payload_from_any_peer(&evm
let expecting_payload = self.execution_payload_cache.is_waiting_for_payload(&ev
```

```
if  payload_already_seen && !expecting_payload {
return Ok(());
}


// 캐시에 추가
self.execution_payload_cache.add_payload_to_cache(sealed_block).await;
Ok(())
}
```

성능 최적화

1. 비동기 처리

논블로킹 데이터 처리:

```
// 블록 처리를 별도 태스크에서 비동기로 실행
tokio::spawn(async  move {
if  let Err(error) = server
.data_handler
.handle_block_header(gossip_request, peer.address.api, source_socket_addr)
.await
{
error!("Failed to process block: {:?}", error);
}
```

```
});

// 즉시 응답 반환
HttpResponse::Ok().finish()
```

2. 배치 처리

트랜잭션 배치 가져오기:

```
// 누락된 트랜잭션들을 한 번에 요청
let missing_txs = self
.api_client
.get_transactions(source_api_address, &missing_tx_ids)
.await?;
```

3. 캐시 최적화

메모리 효율적인 캐시:

- TTL 기반 자동 만료
- LRU 정책을 통한 메모리 관리
- 읽기/쓰기 락을 통한 동시성 제어

오류 처리 및 복구

1. 네트워크 오류 처리

```
match self.pull_data_and_update_the_score(peer, data_request.clone(), peer_list
Ok(Some(data)) => return Ok(data),
Ok(None) => continue, // 피어가 데이터를 가지고 있지 않음
Err(err) => {
warn!("Failed to fetch from peer {}: {}", peer.0, err);
continue; // 다음 피어 시도
}
}
```

2. 데이터 검증 오류

```
fn handle_invalid_data(
peer_miner_address: &Address,
error: &GossipError,
peer_list: &PeerList,
) {
match error {
GossipError::InvalidData(_) => {
peer_list.decrease_peer_score(peer_miner_address, ScoreDecreaseReason::BogusData
}
GossipError::BlockPool(BlockPoolError::BlockError(_)) => {
peer_list.decrease_peer_score(peer_miner_address, ScoreDecreaseReason::BogusData
```

```
}
_ => {}
}
}
```

모니터링 및 메트릭스

1. 가십 통계

전파 성능 추적:

- 브로드캐스트 완료 시간
- 피어별 응답 시간
- 데이터 중복 비율
- 네트워크 처리량

2. 디버깅 정보

상세한 로깅:

```
debug!(
"Node {}: Broadcasting data to peers: {}",
self.client.mining_address, message_type_and_id
);

debug!(
"Node {}: Peers selected for broadcast: {:?}",
```

```
self.client.mining_address, peers

);
```

이 가십 프로토콜 구현은 Irys 체인의 분산 특성을 지원하며, 효율적인 데이터 전파와 네트워크 복원력을 제공합니다.